The FLOPs You Save Are Not the Time You Spend: Sparse Attention in Multi-Agent Communication

Anonymous authors
Paper under double-blind review

Abstract

Attention-based inter-agent communication aggregates over all $N-1$ peers per agent per step, and restricting each agent to its top-k peers by attention score is an appealing way to make that cheaper. The case for it is normally made by counting operations rather than by measuring time. We set out to build such a mechanism and report why we abandoned it, at three levels.

First, an analytic ceiling. Because selecting the top-k peers *by attention score* requires computing all N scores, only the value aggregation is sparsifiable, and the reduction in attention FLOPs is bounded above by $2\times$ for any k and any N — not the $1-k/(N-1)$ that the aggregation term alone suggests. Sparsity whose pattern is fixed before any score is computed escapes this bound and reaches N/k; data-dependent top-k cannot.

Second, what executes. Applying top-k as a *mask* on the dense attention matrix, as our own first implementation did, leaves the dense aggregation matmul in place, so the executed operation count is identical to dense while a FLOP model based on $1-k/(N-1)$ reports a 91% saving. We implement the genuinely sparse gather, verify it computes the same function while executing fewer operations, and benchmark dense, masked and gathered against a fused baseline.

Third, what it costs. Across a head-width $\times$ batch $\times$ $N \times k$ sweep on one GPU architecture, the gathered path is slower than unfused dense in all 46 cells and slower than the fused kernel in 45; the ordering also holds on a CPU whose machine balance differs by more than an order of magnitude, where it is 2.0–3.1$\times$ slower than unfused dense. A component decomposition and a roofline analysis with measured machine balance explain why: selection and gather move memory for no arithmetic, the sparse contraction is slower than the dense matmul it replaces, and at N=512 the sparse path moves 3.8$\times$ the traffic for 0.6$\times$ the arithmetic. Removing arithmetic from a memory-bound kernel does not save time. We tune the sparse path where it is cheap to do so, and report what remains on the table: its selection kernel still runs at under a tenth of the bandwidth the same device achieves elsewhere, and closing that gap entirely would not reverse the ordering.

A pre-registered training study confirms the channel carries information, while six pre-registered sparsification contrasts are under-powered nulls at five seeds and are reported as such. We release the harness, the corrected implementation, and seven reproducible benchmarking pitfalls that produced spurious results during this study.

1 Introduction

Cooperative multi-agent reinforcement learning systems increasingly pass learned messages between agents, and attention is the dominant mechanism for deciding what to listen to (Das et al., 2019; Jiang & Lu, 2018; Jiang et al., 2020). Because every agent attends to every other, the channel costs $\mathcal{O}(N^2)$ per step, and a natural proposal follows: restrict each agent to its top-k peers by attention score, with $k \ll N$. The efficiency case is easy to state — if dense aggregation touches $N-1$ peers and sparse aggregation touches

k, a $1 - k/(N-1)$ fraction of the work disappears — and easy to accept, because it is an operation count and operation counts are cheap to compute and hard to argue with.

This paper is a report on why we abandoned that design. We set out to build score-based top-k communication for multi-agent RL on exactly the rationale above. We are not aware of a published multi-agent communication method that selects peers by attention score after computing all scores; the restriction mechanisms in that literature are largely structural, and, as we show below, structural selection is not subject to the limit we derive. The argument we examine here is therefore one we made ourselves, and the implementation errors we report are our own. We think the result is worth reporting anyway, because the reasoning that led us there is standard, the failure has three distinct causes at three levels of abstraction, and only the third is specific to our hardware.

The count is smaller than it looks. Selecting the top-k peers by attention score requires computing every score. The score matrix is therefore irreducible, and only the value aggregation — half of attention — can be sparsified at all. Section 3 states this as a proposition: the achievable reduction in attention FLOPs is bounded above by $2\times$, for every k and every N, approached only as $k/N \to 0$. The familiar $1 - k/(N-1)$ describes one of two equal terms. The bound has a precise escape route, which we make explicit because it is the design target for any successor method: if the sparsity pattern is structural — fixed by a topology, a spatial prior, or a hash computable in $o(N^2 d)$ — the score matmul becomes sparsifiable too and the ratio reaches N/k, unbounded.

Even the reduced count need not be executed. A count is a model of cost, not a measurement of it, and an implementation can fail to perform the saving the count describes. Applying top-k as a *mask* on the dense attention matrix materialises the discarded entries as zeros and runs the same dense $(B, N, N) \times (B, N, d)$ aggregation matmul; executed multiply–accumulates are then identical to dense. This is what our own first implementation did, and a FLOP model based on $1 - k/(N-1)$ reports a 91% saving for it. We implement the genuinely sparse gather instead, verify it computes the same function to floating-point tolerance while executing measurably fewer FLOPs, and benchmark dense, masked and gathered as three separate curves against a fused dense baseline.

And the executed saving need not be time. The gathered implementation is slower than unfused dense in all 46 cells of a head-width $\times$ batch $\times$ N sweep, and slower than a fused kernel in 45 of them. Section 4 decomposes why: the sparse contraction is itself slower than the dense matmul it replaces, because a batched matrix–vector product has far lower arithmetic intensity than a real GEMM; selection and gather move substantial memory for no arithmetic; and at N=512 the sparse path moves $3.8\times$ the traffic for $0.6\times$ the arithmetic. Removing arithmetic from a memory-bound kernel does not make it faster. We name the dense baseline and the envelope with every number, because the two baselines differ by up to $4.6\times$ and which one is meant sometimes changes the answer.

Contributions.

- **Analytical.** A $2\times$ ceiling on the FLOPs saving achievable by any score-based top-k attention, with its assumptions separated and its structural escape route stated as a constructive proposition (Section 3).
- **Implementation.** A masked/gathered/fused taxonomy of the same function, a corrected sparse implementation, and regression tests pinning that the two top-k variants agree numerically while executing different work (Sections 3, 4, Appendix A.3).
- **Systems evidence.** A benchmark campaign over $N \leq 512$, head width 16–256, batch 32–4096, two dtypes, three execution modes and two architectures, with a component decomposition and a roofline account of the cause, and a drift control quantifying how far the measurement travels (Section 4).

- **Learning evidence, bounded by its power.** A pre-registered study separating "the channel does nothing" from "sparsifying the channel is free", with positive controls that pass and six sparsification contrasts reported as the under-powered nulls they are (Section 5).

- **Methodological artifacts.** Seven reproducible benchmarking pitfalls, each of which produced a spurious result during this study, and a released harness that regenerates every table here from raw artifacts (Section 6).

Every claim concerns centralised-inference computational cost on one device, not bandwidth-constrained distributed deployment, where the objective is to reduce messages physically transmitted and where top-k sparsity may well be the right design. Section 7 states the full envelope, including where our own implementation is known to leave performance on the table.

2 Related Work and a Reporting Audit

2.1 Attention-based inter-agent communication

CommNet (Sukhbaatar et al., 2016) broadcasts mean-pooled hidden states; TarMAC (Das et al., 2019) introduces signature-based targeting so that receivers weight senders; IC3Net (Singh et al., 2019) gates whether an agent speaks at all; ATOC (Jiang & Lu, 2018) forms bounded communication groups; DGN (Jiang et al., 2020) restricts attention to a spatial neighbourhood; MAT (Wen et al., 2022) casts the joint policy as a sequence model. The pattern persists in applied systems: MARL-IoTP (Hamdan et al., 2026) pairs heterogeneous MAPPO agents with an attention-based message aggregator for edge orchestration, and ablates that aggregator against mean pooling. Several of these works motivate restricted communication with a cost argument. DGN, for instance, observes that receiving a large amount of information "requires high bandwidth and incurs high computational complexity" (Jiang et al., 2020) — a claim about both transmitted bytes and local compute, of which only the second is what this paper measures.

Two points about this group matter for what follows. First, the restriction mechanisms are mostly *structural*: DGN's neighbourhood is spatial and ATOC's groups are formed before attention weights are computed, so both fall under Proposition 2 rather than Proposition 1. Second, we did not find a method in this literature that selects peers by attention *score* after computing all scores, which is the mechanism this paper analyses. We flag that explicitly because it determines what our result does and does not bear on: we are evaluating a design we proposed and built, and the ceiling of Section 3.2 constrains that design, not the structural selectors above.

2.2 Efficient and sparse attention

The efficient-attention literature is where the operation-count argument for sparsity is most developed, and where the structural-versus-data-dependent distinction is sharpest. Longformer (Beltagy et al., 2020) and BigBird (Zaheer et al., 2020) fix a sparsity pattern in advance — sliding windows, global tokens, random edges — so the score computation itself becomes sparse and the saving is asymptotic. Reformer (Kitaev et al., 2020) obtains its pattern from a locality-sensitive hash computed without forming all pairwise scores; Performer (Choromanski et al., 2021) avoids the score matrix altogether through a kernel factorisation. All of these escape the ceiling of Proposition 1 for the same reason: none of them needs the complete score matrix in order to decide what to keep.

Sparsemax (Martins & Astudillo, 2016) and α-entmax (Peters et al., 2019) obtain exact zeros from the normalisation rather than a hard cutoff, but do so *after* the scores exist and are therefore subject to the ceiling; Gumbel-softmax (Jang et al., 2017) provides the relaxation used by learned discrete gates. Constrained-RL formulations (Achiam et al., 2017; Stooke et al., 2020) supply the machinery for enforcing a communication budget rather than trading it against return at a fixed weight. We implement entmax and a Lagrangian budget as arms in Section 5.

Table 1: Type of evidence offered for efficiency claims, in the twelve audited papers (n=6 per group). "Reports latency" requires a measured temporal quantity attributable to the mechanism. This is an audit of reporting practice in a small, deliberately selected set — the papers we ourselves relied on — not a sample of either literature, and a "no" means the argument rests on operation counts or asymptotics, not that it is wrong.

Paper	**Efficiency claim**	**Type**	**Reports latency**
Multi-agent communication / attention			
CommNet (Sukhbaatar et al., 2016)	none explicit	—	n/a
TarMAC (Das et al., 2019)	compact messages	qualitative	no
IC3Net (Singh et al., 2019)	fewer messages	counts	no
ATOC (Jiang & Lu, 2018)	bounded group size	counts	no
DGN (Jiang et al., 2020)	bandwidth + complexity	asymptotic	no
MAT (Wen et al., 2022)	linear in N	asymptotic	**yes**
Efficient attention / sparse mixture-of-experts			
Reformer (Kitaev et al., 2020)	$\mathcal{O}(L \log L)$	asymptotic	**yes**
Longformer (Beltagy et al., 2020)	$\mathcal{O}(L)$	asymptotic	**yes**
BigBird (Zaheer et al., 2020)	$\mathcal{O}(L)$	asymptotic	no
Performer (Choromanski et al., 2021)	$\mathcal{O}(L)$	asymptotic	**yes**
Switch (Fedus et al., 2022)	FLOPs-matched	FLOPs	**yes**
Sparse MoE (Shazeer et al., 2017)	large capacity, low cost	FLOPs	**yes**

2.3 Sparse routing and mixture-of-experts

Score-based top-k selection is used at scale in sparse mixture-of-experts, where a router scores every expert and dispatches each token to the best k (Shazeer et al., 2017; Fedus et al., 2022). The structure of the argument is the same one we examine, and so is the exemption: routing scores are cheap relative to the expert FLOPs they gate, so the analogue of our N^2d score term is negligible there, and the saving is real. That contrast is instructive. Whether score-based selection pays depends on the ratio between the cost of scoring all candidates and the cost of the work being skipped — large in MoE, and equal to one in single-head attention, which is exactly Proposition 1.

2.4 Measuring efficiency rather than counting it

That operation counts, memory traffic, kernel fusion and realised latency are distinct quantities is established in the systems literature and in parts of the attention literature. The roofline model (Williams et al., 2009) formalises when arithmetic ceases to govern runtime. FlashAttention (Dao et al., 2022) and memory-efficient attention (Rabe & Staats, 2021) make the point from the other direction: the winning implementations of dense attention reduce memory traffic without changing the operation count at all, which is precisely the axis a FLOPs-based argument cannot see.

One precedent deserves particular mention because it anticipates the distinction this paper turns on. Longformer reports runtime and memory for *three* implementations of the same attention pattern: a loop version, a vectorised "chunks" version, and a custom CUDA kernel. The authors note that the chunked variant consumes roughly twice the optimal memory because it computes some zero values — that is, they measured a masked implementation separately from a genuinely sparse one, and reported the difference.

A reporting audit. Motivated by that precedent, we checked whether efficiency claims in the papers we drew on are accompanied by a measurement. For each paper we read the full text and recorded (i) whether it makes a computational-efficiency or complexity claim about its mechanism and (ii) whether it reports any measured temporal quantity — time, speed, throughput, or achieved FLOP/s — attributable to that mechanism. A statement such as "training took a few days" does not qualify. Table 1 reports the result.

Among the five multi-agent papers making an efficiency claim, one reports a measured temporal quantity for its mechanism; among the six efficient-attention and mixture-of-experts papers, five do. We draw no

conclusion about either field from twelve papers. The asymmetry is worth stating only because it describes the evidence base we ourselves worked from when we adopted the operation-count argument, and because the argument does not transfer cleanly in any case: the efficient-attention results hold at sequence lengths of 10^3–10^5, where attention is genuinely the bottleneck, while multi-agent communication operates at $N \lesssim 10^2$, where it is not.

Empirical rigour in MARL. Our training protocol follows the recommendations of Papoudakis et al. (2021) for MARL benchmarking, and we report effect sizes and intervals rather than bare means. Our environments come from the PettingZoo/MPE family (Terry et al., 2021), and our learner is MAPPO (Yu et al., 2022).

3 What Sparsifying Attention Can and Cannot Save

3.1 Setting and three implementations of one function

Consider one round of scaled dot-product attention over N agents, per batch element, with head width d. Agent i forms a query q_i, and every agent contributes a key and a value; self-attention is masked. Write $s_i = q_i K^\top \in \mathbb{R}^N$ for the score vector and $a_i = \mathrm{softmax}(s_i)$ for the attention weights. Dense attention computes $m_i = a_i V$.

A top-k variant keeps only the k largest entries of a_i and renormalises. The same function admits implementations with very different costs, and the distinction between them is the one this paper turns on.

Masked top-k. Compute a_i in full, multiply by a $\{0,1\}$ mask, renormalise, and evaluate $m_i = \tilde{a}_i V$ with $\tilde{a}_i$ still dense in memory. The discarded entries are materialised as zeros. Executed multiply–accumulates are identical to dense, plus the cost of selection and masking.

Gathered top-k. Take the top-k of the raw scores, softmax over those k values only, gather the corresponding value rows into a (B, N, k, d) tensor, and contract $(B{\cdot}N, 1, k) \times (B{\cdot}N, k, d)$. No zeros are materialised and the aggregation genuinely shrinks.

Fused dense. Compute the same function as dense through a fused kernel (Dao et al., 2022; Rabe & Staats, 2021) that never materialises the (B, N, N) weight matrix.

Figure 1 sets the three side by side. Gathered and masked top-k compute the same function: softmax is monotone within a row, so the top-k weights are the top-k scores, and a softmax over just those k scores equals the dense softmax restricted to them and renormalised. We verify agreement to $< 10^{-10}$ in `float64` and pin it with a regression test (Appendix A.3). The operation-count argument is about the gathered variant; timing the masked variant measures an implementation artifact, and timing either against unfused dense alone ignores what a production kernel costs. We therefore report all three throughout, each against both dense baselines.

3.2 A ceiling on what sparsity can save

Counting multiply–accumulates in the two matrix products that constitute attention, and excluding the shared input and output projections:

$$C_{\text{scores}} = N^2 d, \quad C_{\text{agg}} = N^2 d, \quad C_{\text{dense}} = 2N^2 d, \qquad C_{\text{top-}k} = \underbrace{N^2 d}_{\text{all scores}} + \underbrace{Nkd}_{\text{aggregate } k}. \tag{1}$$

Proposition 1 (Ceiling). *For any selection rule that requires the complete score vector of each query,*

$$\frac{C_{dense}}{C_{top\text{-}k}} = \frac{2N^2 d}{N^2 d + Nkd} = \frac{2N}{N+k} < 2 \qquad \textit{for all } k \geq 1, \tag{2}$$

with supremum 2 approached only as $k/N \to 0$. No choice of k or N yields more than a $2\times$ reduction in attention FLOPs.

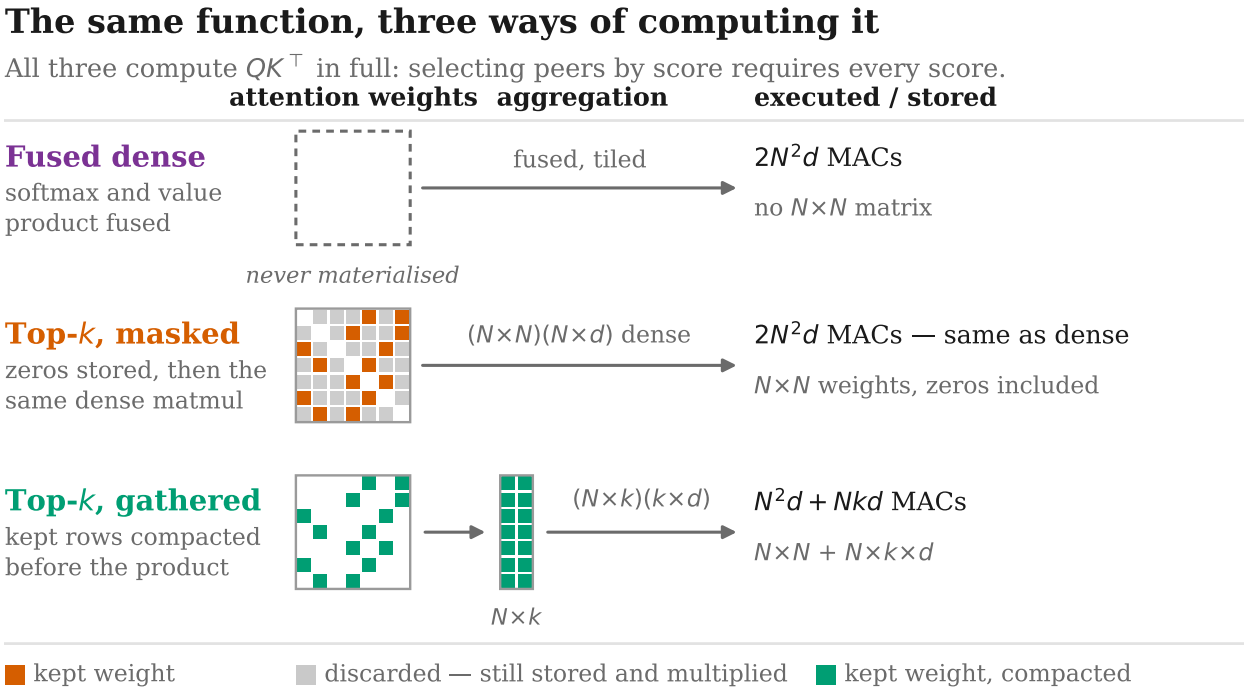


Figure 1: The same top-k attention, computed three ways. All three form the full score matrix, because selecting peers by score requires every score. They differ in what happens next: the fused kernel never materialises the weights; the masked variant stores the discarded entries as zeros and multiplies them anyway, so its executed operation count equals dense; only the gathered variant compacts the kept rows and shrinks the product. The efficiency argument examined in this paper is an argument about the third row.

Proof. $C_{\text{top-}k} > N^2d = \frac{1}{2}C_{\text{dense}}$ for every $k \geq 1$ since $Nkd > 0$. The ratio decreases monotonically in k and tends to 2 as $k/N \to 0$. □

We use N^2d rather than $N(N-1)d$ because it is the count the implementations actually execute: self-masking is applied by writing $-\infty$ into the diagonal of a full $N \times N$ score matrix, not by skipping work. Counting the masked-out diagonal exactly instead would give $2(N-1)/\big((N-1)+k\big)$, which is strictly smaller than $2N/(N+k)$ for every $1 \leq k \leq N-1$. Proposition 1 is therefore a valid upper bound under either convention, and a slightly loose one under the exact count. The figure $1 - k/(N-1)$, which describes the aggregation term alone, is at most half of attention; read as *the* saving it overstates the achievable reduction by a factor approaching two, before any question of whether it is realised in time.

The assumptions are worth separating, because only one of them is load-bearing. **(A1) Single round**: with R rounds both terms scale by R and the ratio is unchanged. **(A2) Data-dependent selection**: the kept set is a function of the scores, so all N scores must be formed. **(A3) Per head**: with H heads of width d/H both terms scale identically. **(A4) Projections excluded**: including the shared qkv and output projections moves the ratio strictly closer to 1, so the bound remains valid. **(A5) Selection cost excluded**: counting the top-k operation itself can only decrease the ratio, and Section 4.4 shows this term is large in practice.

A2 is the assumption that carries the result. It is a statement about a class of selection rules, not about sparsity in general, and when it fails the ceiling lifts.

Proposition 2 (Escape)**.** *If the kept set is determined* structurally *— by information obtainable without forming the scores, in $o(N^2d)$ — then scores need not be computed for discarded pairs, and $C_{\text{struct}} = Nkd + Nkd = 2Nkd$, giving $C_{dense}/C_{\text{struct}} = N/k$, which is unbounded in N/k.*

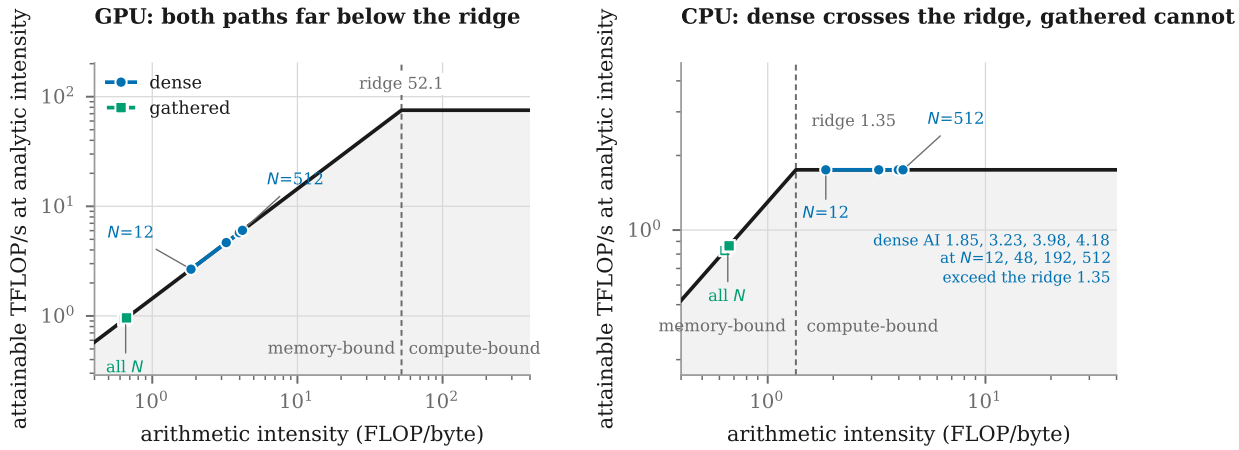


Figure 2: Roofline with measured machine balance. Marker heights are the roof evaluated at each path's analytic intensity, not achieved throughput. On the GPU (ridge 53.0) both paths lie deep in the memory-bound region at every N. On the CPU, whose balance is more than an order of magnitude lower, the dense path's intensity saturates at $(4d{+}5)/16 = 4.31$ and crosses the measured balance for $N \gtrsim 96$, while the gathered path stays pinned near 0.66 and remains memory-bound throughout.

Qualifying rules include a fixed communication topology, a spatial or distance prior, a locality-sensitive hash, and a learned *static* graph. This is why Longformer and BigBird obtain asymptotic savings where data-dependent top-k cannot (Beltagy et al., 2020; Zaheer et al., 2020): their patterns are fixed before any score is formed. It is also the concrete design target for a successor method — to exceed 2×, a method must decide where to look before looking. In an environment with spatial structure a distance-calibrated selector is the obvious candidate.

Proposition 2 concerns operation counts only. It says that structural selection removes the assumption responsible for the ceiling; it does not say that any particular structural method attains a proportional speedup, which is a separate question and one the rest of this paper suggests should be asked separately.

3.3 Why the ceiling is not the binding constraint

Proposition 1 bounds a quantity that, in this regime, does not govern runtime. For memory-bound kernels time tracks bytes moved rather than arithmetic performed (Williams et al., 2009), and the two paths differ far more in bytes than in FLOPs.

We compute arithmetic intensity AI = FLOPs/bytes analytically per stage, counting each tensor read and written once, and compare it against a *measured* machine balance: a large GEMM gives achievable FLOP/s, a large device-to-device copy gives achievable bandwidth, and their ratio is the ridge point above which a kernel is compute-bound. Two caveats apply to this model and we state them before using it. Counting each tensor once ignores cache residency, so at small N — where the whole working set fits in the GPU's 128 MB L2 — the analytic byte counts overstate traffic and the model does not apply. And our CPU bandwidth probe uses buffers smaller than the host's 96 MiB L3, so the CPU ridge is a cache-bandwidth figure and is not reproducible to better than a factor of a few: repeated measurement of the same host gives 1.35 and 3.68 FLOP/byte on different days, and a DRAM-resident probe gives lower still. We therefore use the CPU ridge only for statements that hold across that whole range, and the dense path's saturating intensity of 4.31 is above all of them.

Table 2 and Figure 2 give the result. The gathered path adds two stages that move substantial memory for no arithmetic — the top-k selection and the gather — so its intensity is pinned near 0.66 and does not improve with N. Dense intensity, by contrast, rises towards $(4d{+}5)/16 = 4.31$. At $N{=}512$ the sparse path

Table 2: Measured machine balance, and analytic arithmetic intensity of the two paths at batch 256, d=16, $k = 25\%$ of peers. The gathered path is memory-bound on both devices at every N. The dense path is memory-bound on the GPU at every N, and exceeds the CPU balance at every N we measure, under any of the estimates we obtain for it.

Device	**Achievable GEMM**	**Achievable bandwidth**	**Ridge point**
RTX PRO 6000 Blackwell (`fp32`)	75.3 TFLOP/s	1444 GB/s	52.1 FLOP/byte
x86_64 CPU (`fp32`, cache-resident)	1.75 TFLOP/s	1300 GB/s	1.35 FLOP/byte
N	**dense AI**	**gathered AI**	**ratio**
12	1.85	0.64	0.34×
48	3.23	0.66	0.20×
192	3.98	0.66	0.17×
512	4.18	0.66	0.16×

moves 4.16 GB to perform 2.76 GFLOP, against dense's 1.11 GB for 4.63 GFLOP: 3.8× the traffic for 0.6× the arithmetic.

Where the model's assumptions hold it predicts well. At $N \in \{192, 512\}$, where the working set exceeds L2 and traffic is real, both paths attain a substantial fraction of achievable bandwidth and the ratio of analytic bytes tracks the ratio of measured latencies. At $N \leq 48$ it does not apply, and Section 4.2 shows those cells are governed by kernel launch count instead.

The quantity this analysis bounds is a ratio of intensities — roughly 0.16 at large N — which is a property of the two algorithms rather than of any device constant, so the qualitative disadvantage should be expected to persist wherever the gathered path stays below the relevant machine balance. That is not by itself evidence that it does: an argument constructed to be device-independent cannot demonstrate device-independence. Section 4.6 supplies measurements on a second architecture, and Section 7 states what they establish.

4 Measuring the Cost

4.1 Protocol

We time the communication module in isolation, with the training loop removed. The protocol is stated in full because several of its provisions were forced by defects that produced spurious results first (Section 6).

Timing uses on-device CUDA events with an explicit synchronisation on the correct device. Any diagnostic that calls `.item()` is disabled, because such a call synchronises the device *inside* the timed forward pass. Arms are timed interleaved repetition by repetition rather than in sequential per-arm blocks, so that interference from other tenants lands on all arms at once and the ratios between them remain paired; on this shared host the sequential design produced swings above 100% between identical configurations, which interleaving reduced to typically under 2%. We report min over samples as the primary statistic, since contention can only add time, and give median, IQR and inter-run spread in Appendix A.1 together with the sensitivity of every headline ratio to that choice. GPU clocks could not be locked — the account lacks the privilege. Executed FLOPs are counted with `torch.utils.flop_counter` so that latency is always reported beside the arithmetic actually performed.

Unless stated otherwise: PyTorch 2.11 / CUDA 12.8, `fp32`, input embedding width 64, head width d=16, batch 256, $k = 25\%$ of peers, and an idle device.

4.2 Three curves

Table 3 is the central measurement. The gathered implementation does execute fewer multiply–accumulates than unfused dense — 3553 against 5167 MFLOP at N=512, a 31% reduction consistent with Proposition 1 — and it is nonetheless 3.28× slower than unfused dense and 15.18× slower than the fused kernel. Correcting the implementation helps without being sufficient: replacing the masked variant with the gathered one

Table 3: Forward latency (ms, min over interleaved samples) of three implementations of the same function, plus the fused dense baseline, on one NVIDIA Blackwell GPU in `fp32` at batch 256, d=16. The gathered path executes fewer FLOPs than unfused dense at every N and is slower than both dense baselines at every N. Ratios are gathered divided by each dense baseline.

		dense		**top-k**		**ratio**		**MFLOP**
N	k	fused	unfused	MASKED	GATHERED	/unfused	/fused	gath. vs dense
6	1	0.120	0.127	0.175	0.146	1.15	1.22	10.6 vs 10.8
12	2	0.121	0.127	0.175	0.148	1.16	1.22	21.8 vs 22.8
24	5	0.102	0.123	0.171	0.144	1.17	1.41	46.6 vs 50.3
48	11	0.110	0.131	0.180	0.151	1.15	1.38	105 vs 120
96	23	0.106	0.127	0.198	0.205	1.61	1.93	257 vs 315
192	47	0.151	0.206	0.503	0.596	2.90	3.95	703 vs 931
384	95	0.290	1.157	3.149	2.844	2.46	9.82	2161 vs 3070
512	127	0.431	1.991	7.057	6.534	3.28	15.18	3553 vs 5167

improves the ratio against unfused dense from $\approx 1.40\times$ to $\approx 1.16\times$ for $N \leq 48$. The improvement does not persist — at N=96 and N=192 the gathered path is slower than the masked one it corrects (0.205 vs 0.198; 0.596 vs 0.503), because the gather's traffic grows faster than the masked path's wasted arithmetic.

The two ends of this table are governed by different mechanisms, and the distinction matters for how far the numbers generalise. For $N \leq 96$ the unfused dense arm is nearly flat at 0.12–0.13 ms while its work grows 256-fold: these cells are bound by kernel launch count, not by bandwidth or arithmetic, and the 1.15–1.17× ratios there reflect how many kernels each arm issues. Ratios in this regime are sensitive to how many arms share a timing group, and we measure a 5× disagreement between two of our own sweeps on the same nominal cell for this reason (Section 6, P7). From N=192 upward the working set exceeds L2, traffic is real, and the roofline account of Section 3.3 applies.

Robustness. The ordering is unchanged under `torch.compile` and CUDA-graph capture: at N=512 the ratio against unfused dense is 3.29, 3.30 and 3.31 for eager, compiled and graph-captured execution respectively. It is also unchanged at batch 4096, where the largest cells become FLOP-bound. In `bf16` the gap is wider, reaching 19.7× the fused kernel at N=512. We note that FlashAttention never served any arm: our fused baseline passes an additive float mask, which excludes that backend at every dtype, so the fused numbers throughout are PyTorch's memory-efficient CUTLASS kernel. A FlashAttention baseline reached through a Boolean mask would be a stronger comparison than the one we report, not a weaker one.

Selection is tuned, and it matters how much. The gathered path calls `torch.topk` with `sorted=False`. The ordering of the k retained entries is irrelevant here, since the softmax and the gather are permuted consistently by the same indices, so the sorted and unsorted calls compute the identical function — we verify agreement to 5.4×10^{-7} — while the unsorted one is strictly cheaper. The difference is not small on the stage that dominates this path: with the default `sorted=True` the selection stage costs up to 35% more and the whole gathered arm 5–24% more, which at N=192 is the difference between a ratio of 2.90 and one near 3.8. We report the tuned configuration throughout, because the question is whether a competent sparse implementation wins, not whether a careless one loses. The size of that gap is itself part of the finding: a sparse path has more places to leave performance on the floor than a dense one, and an operation count sees none of them.

4.3 Is there any (d, B, N, k) where sparsity wins?

Reporting isolated favourable cells as anomalies is not a characterisation. We swept head width $d \in \{32, 64, 128, 256\}$ against batch $\in \{32, 256, 4096\}$ and $N \in \{12, 48, 192, 512\}$, and for each cell took the *best* k for the gather path, which is the most favourable operating point available to it.

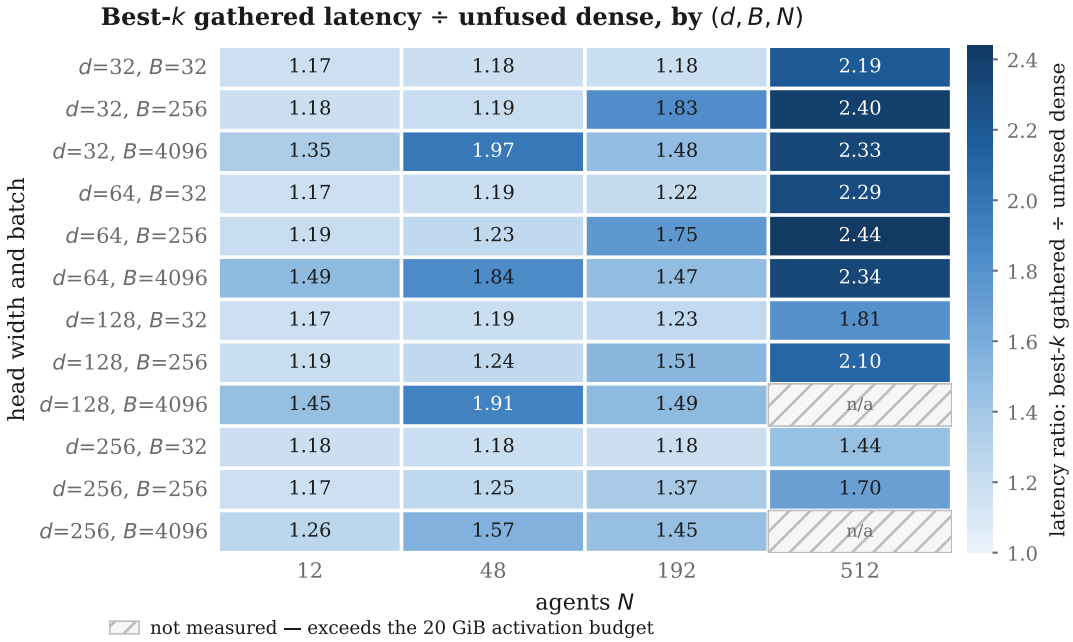


Figure 3: Crossover surface: best-k gathered latency divided by *unfused* dense, on GPU in `fp32` eager mode. Every one of the 46 measured cells exceeds 1. The ratio falls monotonically in head width at N=512 ($2.40 \to 2.44 \to 2.10 \to 1.70$ at B=256), so the surface is flat in batch but not in d; the sweep ends at d=256. Hatched cells exceeded the memory budget. Against the *fused* baseline, one of the 46 cells falls below 1 (Section 4.3).

Gathering beats unfused dense in 0 of 46 measured cells and beats the fused baseline in 1 of 46 — at d=128, B=4096, N=12, ratio 0.86, where the fused kernel's fixed overhead dominates at small N. The underlying numbers are tabulated in Appendix A.2. These 46 cells are a single sweep: GPU, `fp32`, eager mode. The CPU, `bf16`, compiled and CUDA-graph results are separate and smaller sweeps, and we do not pool them into this count.

Across the surface the ratio stays within 1.17–2.44×. It is flat in batch across a 128-fold range, which suggests the outcome is not an artifact of one operating point. It is *not* flat in head width: at N=512 it decreases monotonically as d grows ($2.19 \to 2.29 \to 1.81 \to 1.44$ at B=32), and the sweep stops at d=256 with the two most favourable remaining cells ($d \in \{128, 256\}$ at B=4096, N=512) skipped for memory. We therefore make no claim for $d > 256$, and we regard the head-width trend as unresolved rather than as evidence either way. Head widths above 256 are also well outside the range used in multi-agent communication, which is why the sweep was designed as it was; a reader interested in the large-d limit should treat this as an open question.

4.4 Where the time goes

Table 3 says *that* the sparse path loses; the following says where its time is spent. We time each stage in isolation under the same protocol and form two quantities: the *saving* the sparse contraction buys ($\texttt{agg_dense} - \texttt{agg_sparse}$) and the *overhead* selection costs ($\texttt{topk} + \texttt{gather} + \texttt{softmax}_k - \texttt{softmax}_N$).

Figure 4 and Table 4 give two results, both against the hypothesis they were meant to test. First, the aggregation saving is negative: the sparse $(B{\cdot}N, 1, k) \times (B{\cdot}N, k, d)$ contraction is 3.4× slower than the dense $(B, N, N) \times (B, N, d)$ matmul it replaces at N=512, because a batched matrix–vector product has far lower arithmetic intensity than a real GEMM and cannot use the same hardware paths. Second, selection is the largest single stage: `topk` alone costs 13.7× the entire dense aggregation at N=512, and 51% of the gathered path's forward pass.

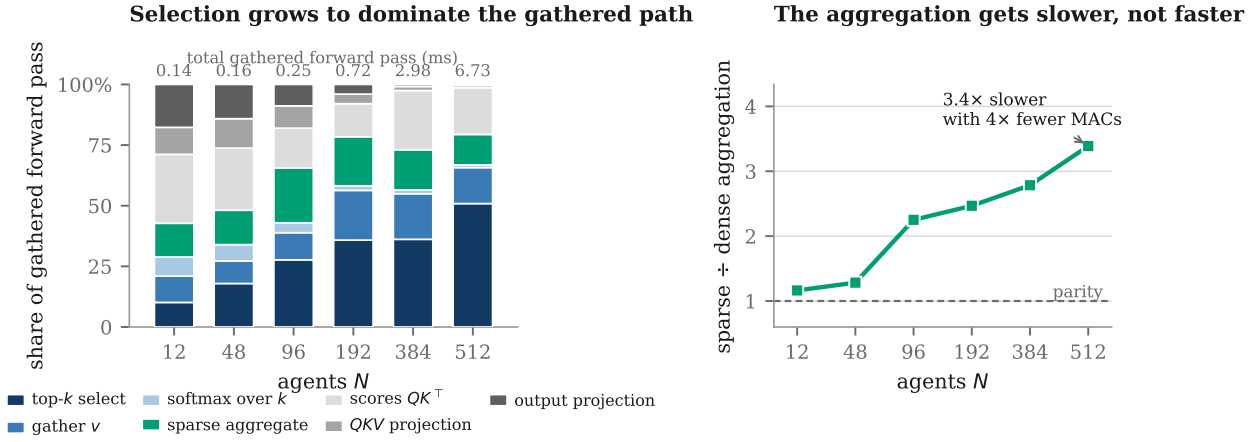


Figure 4: **Left:** how the gathered path spends its time, as a share of its full forward pass (absolute totals above each bar, in ms). Top-k selection grows from 10% to 51% of the budget as N rises, and selection together with the gather and the softmax over k grows from 29% to 67%. **Right:** the aggregation that sparsity was supposed to shrink. The sparse contraction is slower than the dense matmul it replaces at every N, reaching 3.4× at N=512 while performing roughly four times fewer multiply–accumulates.

Table 4: Per-stage latency (ms, min over interleaved samples), batch 256, d=16. The aggregation "saving" is *negative* at every N: the sparse contraction is slower than the dense matmul it replaces, despite performing $\approx 4\times$ fewer multiply–accumulates. Overhead is $\texttt{topk} + \texttt{gather} + \texttt{softmax}_k - \texttt{softmax}_N$ and so is not a sum of the columns shown.

N	k	`topk`	`gather`	`agg_dense`	`agg_sparse`	saving	overhead
12	2	0.014	0.015	0.017	0.019	−0.003	+0.029
48	11	0.028	0.015	0.017	0.022	−0.005	+0.042
96	23	0.069	0.028	0.025	0.056	−0.031	+0.095
192	47	0.258	0.148	0.059	0.146	−0.087	+0.383
384	95	1.078	0.561	0.178	0.496	−0.318	+1.480
512	127	3.426	0.999	0.249	0.844	−0.595	+4.136

How much of this a better kernel could remove is a question the roofline model can answer, and the answer is: a substantial part of the selection term, but not the outcome. At N=512, B=256 the top-k must read the 268 MB score matrix and write values and indices, about 0.47 GB in total. At the 1444 GB/s we measure as achievable that is a 0.32 ms floor against the 3.426 ms observed, so PyTorch's top-k is running at roughly 9% of achievable bandwidth on this shape even unsorted. The gather stage, by contrast, runs near bandwidth. Selection is therefore *not* intrinsically expensive in the way the raw decomposition suggests, and we withdraw any suggestion that it is: dropping the sort already recovered a fifth of it with a one-word change, and an order of magnitude remains on the table. What survives is the ordering: substituting a bandwidth-optimal selection kernel into the N=512 cell gives roughly 3.4 ms for the gathered path against 1.99 ms unfused and 0.43 ms fused. The conclusion does not depend on the selection kernel being bad. It depends on the score matrix having to be formed and ranked at all, which is assumption A2.

4.5 The evaluated learned gate costs $\mathcal{O}(BN^3)$ memory

Our per-agent learned gate over $k \in \{1, \dots, N-1\}$ broadcasts a $(1, 1, K, N)$ cumulative-keep tensor against $(B, N, K, 1)$ gate probabilities with $K = N - 1$, materialising $(B, N, N-1, N)$ before reducing. Activation memory is therefore $\mathcal{O}(BN^3)$ against dense attention's $\mathcal{O}(BN^2)$ and the fused kernel's $\mathcal{O}(BNd)$.

Figure 5 validates the derivation. Fitting peak $\sim N^p$ over $N \geq 48$ with a constant offset removed gives $p = 3.04$ for the gate, against 1.96 for dense, 2.02 for masked, 1.98 for gathered and 1.36 for fused. The

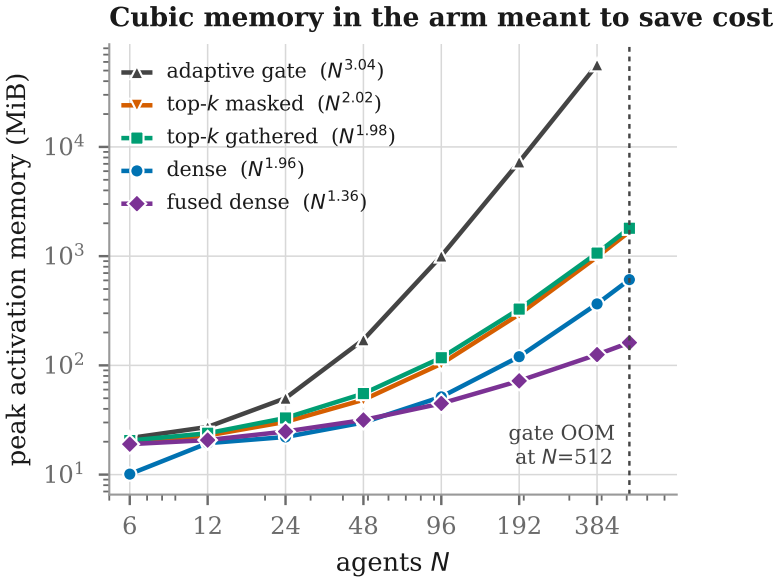


Figure 5: Peak activation memory against agent count, batch 256, with the offset-corrected fitted exponent beside each arm. The evaluated gate implementation grows cubically and is the first arm to exhaust the device; gathering saves no memory over masking, because the gathered value tensor replaces the zeros it avoids.

absolute check is sharper: at N=384 the closed form predicts 97.5% of the measured peak. In practice the gate consumes 55.2 GiB at N=384 (batch 256) and exhausts the device at N=512; analytically it exhausts a 96 GiB card at N=465, and at batch 4096 at N=185.

This is a result about the implementation we evaluated, not a lower bound on learned gates. The same function admits an $\mathcal{O}(BN^2)$ formulation — a cumulative sum along K rather than a broadcast against an explicit cumulative-keep tensor — and we report the cubic scaling as an easy and costly implementation trap rather than as a property of adaptive selection. It belongs with the pitfalls of Section 6, and its interest is that the arm whose stated purpose is to reduce cost was the first to exhaust the device.

4.6 Cross-architecture check

Running the same arms on an x86_64 CPU tests whether the ordering depends on GPU kernel selection. It does not, but obtaining a usable measurement required controlling an effect unrelated to attention. With the default glibc allocator the dense, masked and gathered paths each allocate and free several multi-MiB (B, N, N) temporaries per forward, which the allocator returns to the kernel and which are then re-faulted on the next call: at N=192 this adds $\approx$ 37,000 minor page faults per forward and inflates latency by up to 8.8×. The tax is proportional to the bytes of large temporaries each arm churns, so it differs per arm and does not cancel in a ratio. In our archived CPU run it landed hard enough on the dense arm at N=48 to invert the ordering there — the single cell in this study where a gathered path appeared to beat a dense one on equal terms. It was an allocator measurement, not an attention measurement.

With the heap pinned, the gathered path is slower than both dense baselines at every N: 1.96–3.13× unfused dense and 2.54–28.9× the fused baseline. The N=48 inversion disappears entirely — gathered is 3.13× slower there once the fault storm is removed, against an apparent 0.81× before — and it was the only cell in this study, on either device, where a gathered path appeared to beat a dense one on equal terms. The fused kernel is the arm least affected by the allocator, because CPU flash-attention never materialises a (B, N, N) tensor.

Two things are worth separating here. The gathered-versus-unfused penalty is of the same order on both devices and, at N=192, is smaller on CPU (2.10×) than on GPU (2.90×) — agreement in sign, not in

Table 5: Reproducibility of the headline sweep. Ratios — the quantity every claim is stated in — are far more stable than absolute latency. Entries are median and maximum absolute change across the eight N cells.

	latency $\|\Delta\|$		**ratio $\|\Delta\|$**	
Condition	median	max	median	max
Same device, idle, later run	0.7%	2.6%	**0.4%**	**1.4%**
Same device, concurrent GPU load	67.9%	114.2%	33.7%	115.9%

magnitude. The much larger CPU figure against the *fused* baseline is dominated by a different effect: on CPU the fused kernel's tiling avoids streaming a 37.7 MB score matrix through a 96 MiB L3 shared across two sockets, giving it a 13.8× advantage over unfused dense at N=192 against 4.6× on the GPU. We report that as evidence about which dense baseline matters, not as evidence about arithmetic intensity.

4.7 Drift control

Because clocks are unlocked and the host is shared, we re-ran the identical sweep twice on the same device: once with the device otherwise idle, and once with a concurrent GEMM-and-allocation load running on it.

Table 5 gives the result. Table 5 gives the result. With the device idle the ratios reproduce to within 1.4% at every N and absolute latency to within 2.6%, so the measurement is stable across runs. Under concurrent load agreement degrades sharply — absolute latency moves by a median of 68% — while the ratios, being paired, degrade less. The operational consequence is that headline latency must be measured on an unloaded device, and that interleaving plus minimum-over-samples reduces contention sensitivity without eliminating it.

Direction and magnitude should be reported separately. Under load all eight ratio cells moved against the sparse path, by +12% to +116%. On the idle repeat the movements are small and mixed: five of eight against and three in its favour, none by more than 1.4%. The relevant quantity is therefore the margin rather than the direction. The smallest gathered/unfused-dense ratio we observe on GPU across every run, precision and execution mode is 1.15×; the smallest against the fused baseline is 0.89×, at N=6 and batch 4096, where fixed kernel overhead dominates. On CPU the smallest is 1.96× against unfused dense. No condition we measured moves any GPU cell within 15% of a crossover against unfused dense, and the one contention regime we could measure moves every cell further from one. We cannot exclude an interference pattern that would penalise the dense path more than the sparse one — a competitor that evicted the dense path's working set rather than adding launch latency might do so — but none of the conditions available to us does.

We report a same-device drift control here rather than the second physical card used in an earlier version of this campaign, because the second device was not free of other tenants when these measurements were taken. The idle repeat therefore establishes stability across runs and across hours, not independence from one physical card.

5 Does Sparsification Cost Return?

Sparsification could still be worthwhile if it cost nothing in return. Testing that requires care, because on a task where the channel is not needed "sparsification is free" and "the channel was never used" make identical predictions. We therefore gate the study on a positive control and pre-register the analysis. The study is secondary to the computational result: its role is to establish that the channel being sparsified carries information, and to report what a five-seed design can and cannot resolve about return.

Table 6: Tier-1 positive control (inspected before pre-registration; excluded from the correction) and the six pre-registered contrasts. $A - B$ is the difference of means, HL the Hodges–Lehmann estimate, p the exact two-sided permutation p-value and p_{Holm} its Holm–Bonferroni adjustment over the family of six. C1 is the pre-designated primary contrast.

	Contrast	$A - B$	HL	d	p	p_{Holm}
Positive control						
G1	dense − MAPPO, `simple_reference`	+4.26	—	—	0.008	—
G2	dense − MAPPO, PO `simple_spread` N=6	+11.59	—	—	0.008	—
Pre-registered family						
C1	gathered k=2 − random k=2	+4.03	+4.57	+0.60	0.357	1.000
C2	gathered k=2 − dense	−0.61	−1.98	−0.12	0.865	1.000
C3	gathered k=4 − gathered k=1	+1.76	+1.21	+0.26	0.659	1.000
C4	gathered k=3 − dense, N=12	+147.01	+122.94	+1.27	0.064	0.381
C5	Lagrangian budget − dense	−6.74	−3.39	−0.93	0.183	0.730
C6	entmax − dense	−5.56	−5.44	−1.13	0.119	0.595

5.1 Design and pre-registration

Twelve arms, $N \leq 12$, five seeds, 2M environment steps each, in four priority tiers. All N-scaling is delegated to the training-free measurement of Section 4, because end-to-end wall-clock in this pipeline is environment-bound and cannot attribute a latency difference to the module.

We use two communication-critical controls. In `simple_reference` each agent observes only its partner's goal, so the task requires information to pass between agents; the environment provides its own discrete utterance channel, and our control ablates the *learned attention channel* on top of it rather than removing communication altogether. It is fixed at N=2, where attention entropy is identically zero, so it validates the message pathway rather than the selection mechanism. To carry the k arms we introduce a partially observable `simple_spread`: for agent i and peer j, if $\|\text{peer_rel}[i,j]\|_2 > R$ that entry is zeroed, with R=1.0 and nothing else changed. Under a uniform random policy the mean fraction of peers visible is $0.342, 0.307, 0.263, 0.235$ for $N = 3, 6, 9, 12$, while the absolute number visible rises $0.68 \to 1.54 \to 2.10 \to 2.58$: the world does not grow with N, so density rises and a fixed radius hides a different fraction at each N. That is a difficulty confound across N, and it is why no claim in this paper compares returns across N in this environment. Appendix A.4 gives the remaining characterisation, including a visibility aliasing rate below 0.15% and a 7–19% parameter-count advantage for the communicating arms, which means the positive controls establish that the communicating architecture helps rather than isolating the channel.

The analysis plan was written and committed when 8 of the 40 tier-2–4 runs had produced a final evaluation — all five seeds of `gathered-`k=2 and three of five of `random-`k=2 — and before any tier-2–4 return was read. The tier-1 arms were complete and had been inspected, which is why the tier-1 gate is excluded from the inferential family. Commit history establishes the ordering (tier-1 artifacts 12:18, plan 12:56, tier-2–4 artifacts 19:11 the same day) and the plan was amended only by appending to its deviations log. Timestamps establish ordering; they cannot establish that a file on disk was unread, and we state the plan's independence as a claim about our conduct rather than as something the artifact proves. The plan fixes the endpoint (final greedy evaluation return, one number per seed), the six contrasts and their direction, the test (exact two-sided permutation on the difference of means over all $\binom{10}{5} = 252$ assignments), the effect sizes, the interval (percentile bootstrap, flagged approximate at n=5), and Holm–Bonferroni correction at FWER 0.05. It also fixes the power expectation in advance: the smallest attainable two-sided p is $2/252 = 0.0079$ against a strictest Holm threshold of $0.05/6 = 0.0083$, so at most one contrast can reach corrected significance and only on perfect separation. Any null is under-powered rather than evidence of absence, and no equivalence claim is available because no TOST margin was pre-registered.

5.2 Results

Table 6 reports the outcome.

The channel carries information. Both positive controls separate: $+4.26$ $[+3.15, +5.23]$ on `simple_reference` and $+11.59$ $[+7.13, +16.44]$ on the partially observable task. Both attain $p = 2/252$, the floor of a 5 vs 5 permutation test, reached only when all five seeds of one arm lie above all five of the other, as they do here. Measured attention entropy on the N=6 control is 0.35 nats against a $\ln 5 = 1.61$ ceiling, so attention is peaked rather than uniform. Sparsification is therefore being applied to a channel that demonstrably carries information — the precondition that makes the rest interpretable.

The six contrasts do not resolve a difference. All six return Holm-adjusted $p \geq 0.38$, the outcome the pre-registered power calculation anticipated. We report this as a design that could not resolve effects of this size, not as evidence that the effects are absent, and make no equivalence claim. C1 is the contrast the design exists for: `gathered` and `random-`k are matched on k, on executed FLOPs and on every hyperparameter, differing only in whether peers are chosen by attention score or uniformly at random. Its point estimate favours attention ($+4.03$, HL $+4.57$, $d = +0.60$) but does not approach significance. The design lacked power to detect a selection effect of that size; nothing follows about whether attention selects informatively.

Two results we report but do not interpret. C4 has the one sign opposite to its pre-registered direction, and it should be set aside for a reason independent of its p-value: neither N=12 arm learned the task. A uniform-random policy scores -758.9 under the same evaluation protocol, against -864.9 for the dense arm — worse than random, with two of five seeds below -975 — and -717.9 for the gathered arm. The $+147$ point estimate compares a diverged run with a barely learning one, and we draw no conclusion from it; the N=12 configuration needs a longer budget or a calibrated radius before it can support any contrast. Second, for C4 and C6 the bootstrap interval excludes zero while the permutation test does not. The pre-registration named the permutation test, so it governs, and the direction of the disagreement is the expected one: a percentile bootstrap of a mean at n=5 resamples from a five-point plug-in distribution — only $5^5 = 3125$ distinct resamples exist per arm — so its intervals are systematically too narrow. We keep both on the page so the reader sees the size of the estimated effect as well as the strength of the evidence for it.

Descriptively, and supporting no claim: the Lagrangian arm meets its budget but not by the intended route, its gate collapsing to expected $k \approx 1.01$ within the first 8% of training while the dual multiplier decays to zero and stays there; entmax settles at a support of 1.64 of 5 peers; and attention entropy falls from 0.35 nats over 5 peers to 0.12 over 2, with k=1 forced to 0 by construction, three numbers that sit on different supports and are reported only as an instrumentation check.

6 Seven Pitfalls in Attention Benchmarking

Each of the following produced a wrong number during this study before it was caught. We report them as reusable guidance for anyone benchmarking attention variants, because a benchmarking result is only as trustworthy as the harness that produced it. In every case the symptom was a plausible result rather than an obvious error; Table 7 summarises them.

P1, P3, P6 and P7 are instrumentation errors: they inflate or flatten a measurement and they leave traces — an impossible latency, a non-monotone curve, an unstable repeat. P2 is a design error that lets a shared environment leak into a comparison. P4 and P5 are specification errors about *what is being compared to what*, and they are the dangerous ones, because the resulting numbers are internally consistent, reproducible, and stable under every check the other five would fail. They are answers to a different question than the one being asked. The defence is to state the operation count each arm executes alongside its latency, which makes a mismatch between the claimed and the executed algorithm visible immediately.

One belief of our own is worth correcting. We assumed that standard counters are blind to fused attention kernels and report zero for them, which would make a FLOPs-only comparison unable to see the baseline that

Table 7: Seven ways our harness measured something other than the algorithm.

Pitfall	Symptom before diagnosis	Corrective practice
P1. A diagnostic `.item()` inside the timed region	Dense latency nearly flat in N, read as evidence that the $\mathcal{O}(N^2)$ cost does not manifest	Gate diagnostics behind a flag that is off while timing
P2. Timing arms in sequential blocks	Swings above 100% between identical configurations; a 3.7× discrepancy for one arm at one N	Interleave arms repetition by repetition; report paired ratios
P3. `synchronize()` with no device argument	Every kernel at every size reported the event-resolution floor of $\approx 0.4\,\mu$s — a spectacular speedup	Pin the device and pass it to every synchronisation
P4. Benchmarking a masked implementation as sparse	The "sparse" arm uniformly slower than dense — true of that code, but silent about the method	Assert executed FLOPs differ, and that the sparse path issues no $(B, N, N) \times (B, N, d)$ product
P5. Comparing only against unfused dense	A 4.6× more favourable comparison at N=512 than against the deployed fused kernel	Report both baselines; check which backend serves the shape and dtype
P6. Trusting the allocator on CPU	Up to 8.8× inflation, a non-monotone curve, and one cell where the ordering inverted	Pin the heap when timing on CPU; check minor faults per forward
P7. Changing the composition of a timing group	Two of our own sweeps disagree by 5× on the same nominal cell	Fix the arm set across sweeps that will be compared, and state it

matters. In PyTorch 2.11 that is not so: `torch.utils.flop_counter` reports the fused path correctly, and it was our harness that then added an analytic term on top of the counter's output for that arm, publishing a figure 1.83× too large. The narrower lesson generalises better: instrument the counter, then check the counter. A FLOP model based on $1 - k/(N-1)$ will report a 91% saving for code whose executed operation count is identical to dense, and only a counter attached to the running program will say so.

7 Limitations

What the claim covers. Every cost result here concerns centralised-inference computational cost: the wall-clock latency and memory of executing the communication module on a single device, where all agents' observations are available to one process. It is not a claim about bandwidth-constrained distributed deployment. In that regime the objective is to reduce the number of messages physically transmitted, and top-k sparsification may well be the correct design even though it does not reduce local compute. The analytic ceiling of Proposition 1 says nothing about transmitted bytes.

Measurement envelope. Single-round attention; scaled dot-product attention with the fused backends available in PyTorch 2.11 / CUDA 12.8; input embedding width 64; head width $d \in \{16, 32, 64, 128, 256\}$; $N \leq 512$; batch 32–4096; `fp32` and `bf16`; eager, `torch.compile` and CUDA-graph execution. Hardware: one GPU architecture (NVIDIA Blackwell, sm_120) and one x86_64 CPU. We make no claim outside this envelope. Within it, the head-width trend is unresolved: at N=512 the ratio decreases monotonically in d and the sweep ends at d=256, so we do not know where, or whether, it reaches 1 for larger heads.

Our sparse implementation is not optimal, and we can bound by how much. GATHERED is a straightforward PyTorch implementation using `gather` and a batched contraction, not a hand-written fused kernel. We removed one avoidable cost — `torch.topk` sorts its output by default, which is unnecessary here and was worth 5–24% of the arm — and one remains: PyTorch's top-k still runs at roughly 9% of the achievable bandwidth we measure, so a selection-optimal kernel would recover most of the remaining selection term. Substituting a bandwidth-optimal select into the N=512 cell still leaves the gathered path

near 3.4 ms against 1.99 ms unfused and 0.43 ms fused, so the ordering survives; but a reader should treat our ratios as upper bounds on the penalty, and a fully fused selection–gather–contraction kernel remains the most informative single follow-up to this work. Such a kernel could reduce intermediate traffic, launch overhead and the cost of ranking. It could not avoid forming the score matrix, which is assumption A2 and is what Proposition 1 rests on. We do not claim that no implementation can ever cross over; we claim that none of the ones available to us does, and that the analytic ceiling bounds what any of them could win.

Generality across architectures. The intensity argument of Section 3.3 predicts the same qualitative disadvantage wherever the gathered path stays below the relevant machine balance. That argument is device-independent by construction and therefore cannot itself be evidence of device-independence. Our empirical support is one GPU architecture and one CPU on which the ordering holds with ratios of the same order, plus a same-device drift repeat establishing run-to-run stability. We were unable to replicate on a different GPU generation — the older hardware that produced this project's earlier runs is no longer available. A reader who weights architectural replication heavily should discount the generality accordingly. We also note that the CPU comparison against the *fused* baseline is dominated by cache blocking rather than by arithmetic intensity, so it is weaker cross-architecture evidence for the intensity mechanism than the unfused comparison is.

Two measurement effects that had to be controlled, and one that remains. Our CPU numbers required pinning the glibc heap; without it, page faults on recycled (B, N, N) temporaries inflate arms unequally and can invert an ordering (Section 4.6). Our GPU ridge point reproduces to within 2%, but the CPU one is measured with buffers smaller than the host's L3 and is therefore a cache-bandwidth figure that does not reproduce: repeat measurements of the same host give 1.35 and 3.68 FLOP/byte, and a DRAM-resident probe gives lower still. We use the CPU ridge only where the conclusion holds across that whole range. What remains uncontrolled is that small-N cells are launch-bound, and their measured ratios depend on how many arms share a timing group; two of our own sweeps disagree by 5× on one such cell. Ratios at $N \leq 96$ should be read with that in mind.

Fused baseline. Our fused arm passes an additive float mask, which excludes the FlashAttention backend at every dtype; all fused numbers are PyTorch's memory-efficient kernel. A FlashAttention baseline would be faster, so our fused comparison understates the gap rather than inflating it.

Unlocked clocks. We lacked the privilege to lock GPU clocks. We compensate with interleaved timing, minimum-over-samples reporting, published dispersion (Appendix A.1) and an explicit drift control (Section 4.7), and we report that absolute latency under concurrent load can move by more than 100%. Our drift control is a same-device repeat; it establishes stability across runs and hours, not independence from one physical card, because a second card free of other tenants was not available when these measurements were taken. Ratios are far more stable, and all claims are stated in ratios.

Statistical power of the training study. Five seeds per arm gives a minimum attainable two-sided permutation p of 0.0079 against a Holm threshold of 0.0083 for six contrasts, which detects only perfect separation. Resolving the $d \approx 0.6$ effect suggested by C1 would need roughly 45 seeds per arm at two-sided $\alpha = 0.05$ and 80% power, and roughly 70 at the family-corrected $\alpha = 0.05/6$. We should be direct that this was a design choice rather than a hard constraint: the entire 60-run study consumed 31.9 GPU-hours against a 200-hour guardrail, and a properly powered C1 would have cost about 37.8. We allocated the budget to the measurement campaign instead, which we would do again given that the computational result is the paper's contribution — but the six nulls are a consequence of that allocation, not of an unaffordable experiment, and a reader is entitled to weigh them accordingly.

Environment. The partially observable `simple_spread` variant is our own, documented in Section 5 and released. It carries a difficulty confound across N that we measure and publish, and which is why no across-N return comparison appears in this paper. Its zeroed visibility mask does not distinguish an absent peer from one at the origin, which affects $\leq 0.15\%$ of visible peers in our runs. At N=12 neither trained arm beats a uniform-random policy, so no contrast at that agent count is interpretable (Section 5). Results on MPE tasks

may not transfer to environments with different observation structure, and our positive controls establish that the communicating architecture helps rather than isolating the channel, since the communicating arms carry 7–19% more parameters.

8 Conclusion

We set out to make inter-agent attention cheaper by restricting each agent to its top-k peers, and the efficiency case for doing so rested on an operation count. Examining that count, and then measuring what runs, produced three separate negative results at three levels of abstraction.

The count is smaller than it appears: because selecting peers by attention score requires every score, only half of attention is sparsifiable and the achievable FLOPs reduction is bounded by $2\times$ rather than by $1-k/(N-1)$. The count need not be executed: masking the dense attention matrix leaves the dense aggregation matmul in place, so an implementation written that way performs no saving at all while a FLOP model reports a large one. And the executed saving need not be time: a genuinely sparse gather, which we verified computes the same function while executing fewer operations, was slower than unfused dense in all 46 cells of our GPU sweep and slower than a fused kernel in 45 of them. Selection and gather move memory for no arithmetic, the sparse contraction is slower than the dense one it replaces, and at $N{=}512$ the sparse path moves $3.8\times$ the traffic for $0.6\times$ the arithmetic.

What does not follow is as important as what does. This is not evidence that sparse multi-agent communication is a bad idea. The result is confined to centralised inference on a single device; in bandwidth-constrained distributed settings the objective is the number of messages transmitted, which our analysis does not address and our measurements do not touch. Nor have we shown that no implementation can cross over. One slack in ours we removed — an unnecessary sort in the selection call, worth 5–24% — and one remains and is quantified: the selection kernel still runs at roughly 9% of achievable bandwidth, and closing that gap entirely narrows the margin without reversing it. What survives every correction we could make is the requirement to form and rank the full score matrix, which is the assumption the ceiling rests on.

The constructive half of the result is Proposition 2. A method whose sparsity pattern is fixed *before* the scores are computed — by topology, by a spatial prior, by a hash — is not subject to the ceiling and can reach N/k. This is not a hypothetical class: it is what the structural selectors in the multi-agent literature already do, and what makes the asymptotic savings in Longformer and BigBird real. In an environment with spatial structure, distance-calibrated routing is the natural instance. A fused kernel that eliminates the intermediate traffic between selection, gather and contraction is the other direction worth pursuing, and would settle how much of our measured penalty is implementation and how much is inherent.

The methodological lesson is not specific to multi-agent RL. An operation count is a model of cost, and like any model it can fail in two ways: it can mis-estimate the quantity it describes, and the quantity it describes can fail to govern the outcome anyone cares about. Both happened here, and a third failure — the implementation not executing the counted algorithm — sat between them. The remedy is cheap: report executed operation counts beside measured latency, name the baseline, and state the envelope. We release the harness, the corrected implementation, the pre-registration, and the seven benchmarking pitfalls that produced spurious results along the way (Appendix A.5).

Broader Impact Statement

This work is a measurement and correction study of a computational-efficiency claim. It introduces no new capability and no new data. Its intended effect is to reduce wasted compute in the design of multi-agent communication architectures by discouraging optimisation against an operation count that does not predict runtime.

References

Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. *International Conference on Machine Learning (ICML)*, pp. 22–31, 2017. URL `https://proceedings.mlr.press/v70/`

achiam17a.html.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document Transformer. *arXiv preprint*, 2020. URL https://arxiv.org/abs/2004.05150.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with Performers. *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2009.14794.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-Awareness. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Michael Rabbat, and Joelle Pineau. TarMAC: Targeted multi-agent communication. *International Conference on Machine Learning (ICML)*, 2019. URL https://arxiv.org/abs/1810.11187.

William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

Mohammed Hamdan, Mohamed Cheriet, Mohammed Hamdan, Ahmed Bali, and Fares Ghaleb. MARL-IoTP: Heterogeneous multi-agent learning for perception-aware edge orchestration. In *Fifth International Conference on Pattern Recognition and Artificial Intelligence (PRAI)*, 2026. doi: 10.2139/ssrn.6517702. URL https://ssrn.com/abstract=6517702. Accepted.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-Softmax. *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1611.01144.

Jiechuan Jiang and Zongqing Lu. Learning attentional communication for multi-agent cooperation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. URL https://arxiv.org/abs/1805.07733.

Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1810.09202.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient Transformer. *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/2001.04451.

André F. T. Martins and Ramón Fernandez Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification. *International Conference on Machine Learning (ICML)*, 2016. URL https://arxiv.org/abs/1602.02068.

Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. *Advances in Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2021. URL https://arxiv.org/abs/2006.07869.

Ben Peters, Vlad Niculae, and André F. T. Martins. Sparse sequence-to-sequence models. *Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1504–1519, 2019. doi: 10.18653/v1/P19-1146. URL https://aclanthology.org/P19-1146/.

Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint*, 2021. URL https://arxiv.org/abs/2112.05682.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1701.06538.

Amanpreet Singh, Tushar Jain, and Sainbayar Sukhbaatar. Learning when to communicate at scale in multiagent cooperative and competitive tasks. *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1812.09755.

Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. *International Conference on Machine Learning (ICML)*, pp. 9133–9143, 2020. URL https://proceedings.mlr.press/v119/stooke20a.html.

Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. URL https://arxiv.org/abs/1605.07736.

J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Rodrigo Perez, Caroline Horsch, Clemens Dieffendahl, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2009.14471.

Muning Wen, Jakub Grudzien Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. Multi-agent reinforcement learning is a sequence modeling problem. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.14953.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785. URL https://doi.org/10.1145/1498765.1498785.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative, multi-agent games. *Advances in Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2022. URL https://arxiv.org/abs/2103.01955.

Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2007.14062.

A Appendix

A.1 Dispersion behind every headline number

Table 3 reports min over interleaved samples, on the grounds that contention can only add time. That statistic is trustworthy only if the underlying sample was not wildly dispersed, so we report the median, the interquartile range and the spread of per-run medians for the same cells, and then state in Table 8 how much the headline ratios depend on the choice.

Across the full `fp32`, batch-256 sweep the median-to-minimum ratio runs from 1.00× to 1.06×, the largest being unfused dense at N=192, whose inter-run spread of 5.1% is the widest in the campaign. No headline number therefore rests on an outlying fast sample. Recomputing every headline ratio from medians rather than minima gives 1.15, 1.15, 1.16, 1.14, 1.59, 2.79, 2.45, 3.28 for $N = 6 \ldots 512$, against 1.15, 1.16, 1.17, 1.15, 1.61, 2.90, 2.46, 3.28 from minima: every cell moves by less than 3.6% and all eight remain above 1. No conclusion in the paper depends on the choice of statistic. The complete table for all N and all arms, with the raw per-run medians, is in the released artifacts.

A.2 Crossover surface, tabulated

Table 9 gives the numbers behind Figure 3.

Reading down the d axis at N=512 shows the trend discussed in Section 4.3: 2.19 → 2.29 → 1.81 → 1.44 at B=32 and 2.40 → 2.44 → 2.10 → 1.70 at B=256. The two cells that would continue the trend at B=4096 are the ones that exceeded memory.

Table 8: Forward-pass dispersion, `fp32`, batch 256, d=16, idle device, for the two smallest N in the sweep. Spread is the range of per-run medians across seven repeated timed blocks.

N	arm	min (ms)	median (ms)	IQR (ms)	median/min	spread
6	fused dense	0.1199	0.1239	0.0021	1.03	0.9%
	unfused dense	0.1266	0.1307	0.0031	1.03	0.9%
	MASKED	0.1751	0.1796	0.0044	1.03	0.7%
	GATHERED	0.1460	0.1498	0.0029	1.03	0.4%
	gate	0.3937	0.4036	0.0095	1.03	0.8%
12	fused dense	0.1213	0.1241	0.0019	1.02	0.2%
	unfused dense	0.1271	0.1316	0.0029	1.04	0.4%
	MASKED	0.1751	0.1809	0.0041	1.03	0.3%
	GATHERED	0.1476	0.1514	0.0027	1.03	0.1%
	gate	0.4066	0.4157	0.0097	1.02	0.7%

Table 9: Best-k gathered latency divided by *unfused* dense, GPU, `fp32`, eager. Values < 1 would indicate the gathered path winning; none do. Against the fused baseline one cell falls below 1 (d=128, B=4096, N=12, ratio 0.86). "skip" marks cells whose activation memory exceeded the budget (at d=256, B=4096, N=512 the gathered value tensor alone is ≈273 GiB) and which are recorded as skipped rather than dropped.

d	batch	N=12	N=48	N=192	N=512
32	32	1.17	1.18	1.18	2.19
32	256	1.18	1.19	1.83	2.40
32	4096	1.35	1.97	1.48	2.33
64	32	1.17	1.19	1.22	2.29
64	256	1.19	1.23	1.75	2.44
64	4096	1.49	1.84	1.47	2.34
128	32	1.17	1.19	1.23	1.81
128	256	1.19	1.24	1.51	2.10
128	4096	1.45	1.91	1.49	skip
256	32	1.18	1.18	1.18	1.44
256	256	1.17	1.25	1.37	1.70
256	4096	1.26	1.57	1.45	skip

A.3 Verification of the corrected implementation

The claim that GATHERED and MASKED compute the same function while executing different work is load-bearing, so it is pinned by regression tests rather than asserted:

- **Functional equivalence.** Maximum absolute deviation between the two paths is $< 10^{-10}$ in `float64` for $N \in \{6, 12, 24\}$ and $k \in \{1, 2, 3, N-1\}$.
- **Distinct executed work.** With N=48, k=4, B=8, d=16 and input width 32, a FLOP counter reports 2,555,904 for dense and MASKED (identical), against 2,015,232 for GATHERED. The difference, 540,672, is exactly the aggregation term $2BNd(N-k)$.
- **No dense aggregation in the sparse path.** Tracing every `torch.matmul` confirms GATHERED issues no $(B, N, N) \times (B, N, d)$ product and does issue $(B, N, 1, k) \times (B, N, k, d)$.
- **The score matmul survives in all modes.** Every top-k variant still issues the full $(B, N, d) \times (B, d, N)$ product, which is assumption A2 of Proposition 1 made mechanical.
- **The ceiling holds numerically.** For $N \in \{6, \ldots, 512\}$ the ratio $C_{\text{dense}}/C_{\text{top-}k}$ is verified < 2, and the structural variant of Proposition 2 is verified to track N/k.

- **Selection order is irrelevant.** `topk` with `sorted=False`, which the gathered path uses, agrees with the sorted call to 5.4×10^{-7} in `fp32` at every (N, k) tested, confirming that the softmax and gather are permuted consistently.

A.4 The partially observable environment

The variant used in Section 5 modifies `simple_spread` in one respect: for each agent i and peer j, the relative-position entry peer_rel$[i, j]$ is zeroed when $\|\text{peer_rel}[i, j]\|_2 > R$. Reward, action space, landmark observations, episode length and observation dimension are unchanged, and R=1.0 throughout.

Two properties of the variant should be stated because they bound what it can support. First, the mask is a zeroing rather than an explicit visibility flag, so a peer at relative position exactly $(0, 0)$ is not distinguishable from an absent one; measured under a uniform random policy, 0.06% of visible peers at N=6 and 0.15% at N=12 lie within 0.05 of the observer, which bounds how often the ambiguity can arise. Second, the visible *fraction* falls with N while the absolute number visible rises, because the world size is fixed; the two series are given in Section 5, and the consequence is a difficulty confound that rules out across-N return comparisons in this environment.

The communicating arms carry between 7.2% (N=12) and 19.0% (`simple_reference`) more trainable parameters than the communication-free MAPPO baseline, since the attention module adds parameters that the baseline has no analogue for. The positive controls therefore establish that the communicating architecture outperforms the baseline, not that the channel alone accounts for the difference. Within the pre-registered family the arms share an architecture and the comparison is tighter; C1 in particular holds k, executed FLOPs and every hyperparameter fixed.

A.5 Reproduction and provenance

The measurement campaign is training-free and completes in minutes. The released harness regenerates the tables in this paper from the raw JSON artifacts, and the training arms are driven by generated configuration files with the tier ordering enforced by the runner.

The pre-registration document has two commits: the plan itself, and a later append to its deviations log. The diff between them touches only the deviations section; the endpoint, contrasts, test, effect sizes, interval and correction are byte-identical to the pre-unblinding version. The log records one item, the bootstrap/permutation tension discussed in Section 5.

Two provenance caveats. The aggregate provenance record covers the earlier microbenchmark phases of this project, which ran on different hardware that is no longer available; run-level provenance for the training arms is in the per-run configuration files rather than in that record. The CPU numbers in Section 4.6 were taken with the glibc heap pinned; an earlier campaign that did not do so is superseded and retained in the artifacts with a note. The activation-memory measurement is reported from a run with the whole card free, since a shared-host repeat exhausts memory one N earlier for the gate; the fitted exponents for the other four arms are identical between the two runs.